capital

# Custom Change Policy Selector Development

Software Version 2015.1

February 2016

**© 2016 Mentor Graphics Corporation**
**All rights reserved**.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire

notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

**RESTRICTED RIGHTS LEGEND 03/97**

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

**Contractor/manufacturer is:**
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

**TRADEMARKS:** The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at:
www.mentor.com/terms_conditions/trademarks.cfm.

**End-User License Agreement:** You can print a copy of the End-User License Agreement from:
www.mentor.com/terms_conditions/enduser.cfm.

## Table Of Contents

**Introduction** ..... **2**
**Interfaces** ..... **3**
IXChangePolicySelector ..... 3
**Supporting Interfaces** ..... **3**
**Exercise** ..... **4**
**Guidance** ..... **8**
**Examples** ..... **9**

# Introduction

This document describes the process for developing a *Custom Change Policy Selector plugin* for Capital applications like Capital Logic or Capital Integrator or Capital ModularXC. A Custom change policy selector plugin allows you to choose a change policy from a given set of change policies. If you are interested in developing Custom Change Policy Selection through a plugin, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

The interactive edits of the design objects in capital applications can be controlled through change policies. By default a user can be assigned a change policy in Capital User, so that when the user logs in, he will be restricted / allowed to edit the objects based on the change policy settings. The plugin comes into picture when a customer wants to override the default change policy assigned to the user and use a different one. The plugin is invoked by the application, to get a change policy whenever the current application context changes. An application context provides the current project, current design and current diagram.

The change policy chosen by the plugin is used to control the interactive edits of the design objects. It is only used to control the interactive edits of attributes and properties of the design object. The creation and deletion of design objects are not be controlled by the change-policies.

# Interfaces

### IXChangePolicySelector

This is the main interface of a Change Policy Selector Plugin. This interface has to be implemented to develop a plugin that determines the change policy for the current context of the application.

**Methods:**

IXChangePolicy getSelectedChangePolicy(*IXCurrentContext context, Map<String, ? extends IXChangePolicy> availablePolicies*) :-

Given the various items provided by the context, selects the change policy from the list of policies provided. The parameter context includes the current project, current design, current diagram, currently selected objects and currently opened diagrams. The parameter availablePolicies contains the set of all defined policies in the current project and in system.

The plugin has to return the desired change policy given the input parameters.

Set<IXChangePolicy> getFilteredChangePolicies(IXCurrentContext context, Map<String, IXChangePolicy> availablePolicies);

This method is NOT operational and is kept for future use.

## Supporting Interfaces

This section documents the interfaces that are being used as return types or method parameters in IXChangePolicySelector.

**IXChangePolicy:** This represents the design change policy. Currently this interface helps only to know the name of the design change policy, through getName().

**IXCurrentContext:** Provides the current project, current design, current diagram, currently opened diagrams and currently selected objects.

# Exercise

In this exercise, you will develop a custom change policy selector plugin.

You need to be familiar with Capital Project and the design application where you intend to deploy this plugin.

1) The next step is to create a new custom *Change Policy Selector* plugin.
    a. Ensure that your Java development environment has been set-up. If you have completed the Java plugin exercise then it should already be setup. If you have not, then you should stop this exercise and complete it now.

    b. Open a UNIX/DOS console. Ensure that the environment variables required for Java, ANT and Capital have been setup in this console. Change directory into the following folder within the Capital installation:
       doc/plugin/examples/Java/src/com/example/plugin/changepolicy.

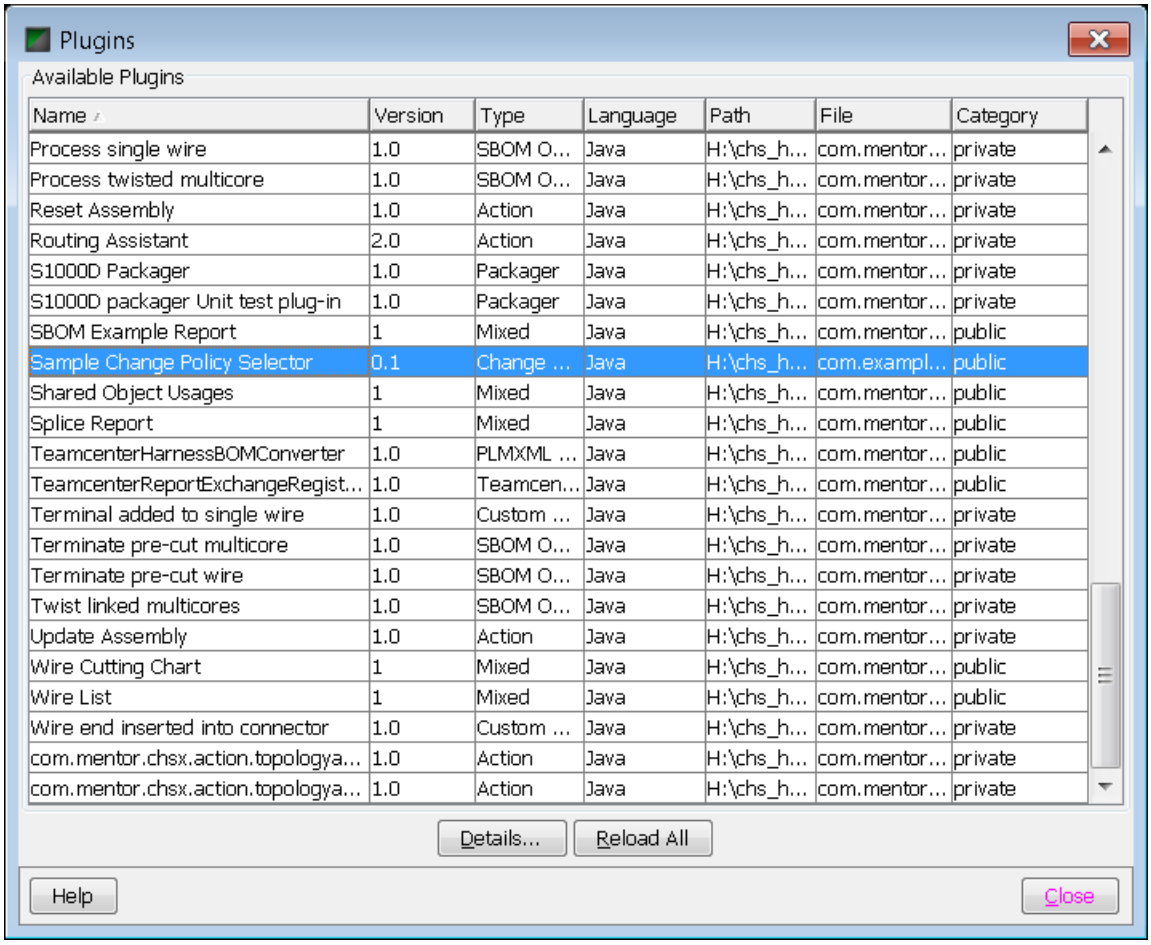
    c. In the above folder, there is an example for Change Policy Selector plugin.

    d. Change directory into “doc/plugin/examples/Java”
    e. Run the command ant ChangePolicySelector
    f. You will see a corresponding .jar file built in plugins/Examples.
    g. If everything is fine, you will eventually get a “BUILD SUCCESSFUL” message. If you do not, you will need to check the environment variables or check for (and fix) errors in the ChangePolicySelector files and try again.

    h. Invoke any design tool for e.g., Capital HarnessXC.

    i. From the main menu, click on the File -> Plugins… menu

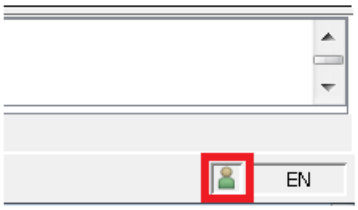
    j. Press the [Reload All] button on the Plugins dialog. You will see your new plugin appear in the list with the other available plugins. See the selected row in the image below:

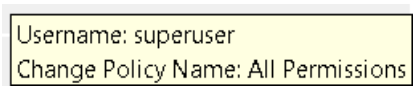



| Name | Version | Type | Language | Path | File | Category |
|---|---|---|---|---|---|---|
| Process single wire | 1.0 | SBOM O... | Java | H:\chs_h... | com.mentor... | private |
| Process twisted multicore | 1.0 | SBOM O... | Java | H:\chs_h... | com.mentor... | private |
| Reset Assembly | 1.0 | Action | Java | H:\chs_h... | com.mentor... | private |
| Routing Assistant | 2.0 | Action | Java | H:\chs_h... | com.mentor... | private |
| S1000D Packager | 1.0 | Packager | Java | H:\chs_h... | com.mentor... | private |
| S1000D packager Unit test plug-in | 1.0 | Packager | Java | H:\chs_h... | com.mentor... | private |
| SBOM Example Report | 1 | Mixed | Java | H:\chs_h... | com.mentor... | public |
| Sample Change Policy Selector | 0.1 | Change ... | Java | H:\chs_h... | com.exampl... | public |
| Shared Object Usages | 1 | Mixed | Java | H:\chs_h... | com.mentor... | public |
| Splice Report | 1 | Mixed | Java | H:\chs_h... | com.mentor... | public |
| TeamcenterHarnessBOMConverter | 1.0 | PLMXML ... | Java | H:\chs_h... | com.mentor... | public |
| TeamcenterReportExchangeRegist... | 1.0 | Teamcen... | Java | H:\chs_h... | com.mentor... | public |
| Terminal added to single wire | 1.0 | Custom ... | Java | H:\chs_h... | com.mentor... | private |
| Terminate pre-cut multicore | 1.0 | SBOM O... | Java | H:\chs_h... | com.mentor... | private |
| Terminate pre-cut wire | 1.0 | SBOM O... | Java | H:\chs_h... | com.mentor... | private |
| Twist linked multicores | 1.0 | SBOM O... | Java | H:\chs_h... | com.mentor... | private |
| Update Assembly | 1.0 | Action | Java | H:\chs_h... | com.mentor... | private |
| Wire Cutting Chart | 1 | Mixed | Java | H:\chs_h... | com.mentor... | public |
| Wire List | 1 | Mixed | Java | H:\chs_h... | com.mentor... | public |
| Wire end inserted into connector | 1.0 | Custom ... | Java | H:\chs_h... | com.mentor... | private |
| com.mentor.chsx.action.topologya... | 1.0 | Action | Java | H:\chs_h... | com.mentor... | private |
| com.mentor.chsx.action.topologya... | 1.0 | Action | Java | H:\chs_h... | com.mentor... | private |

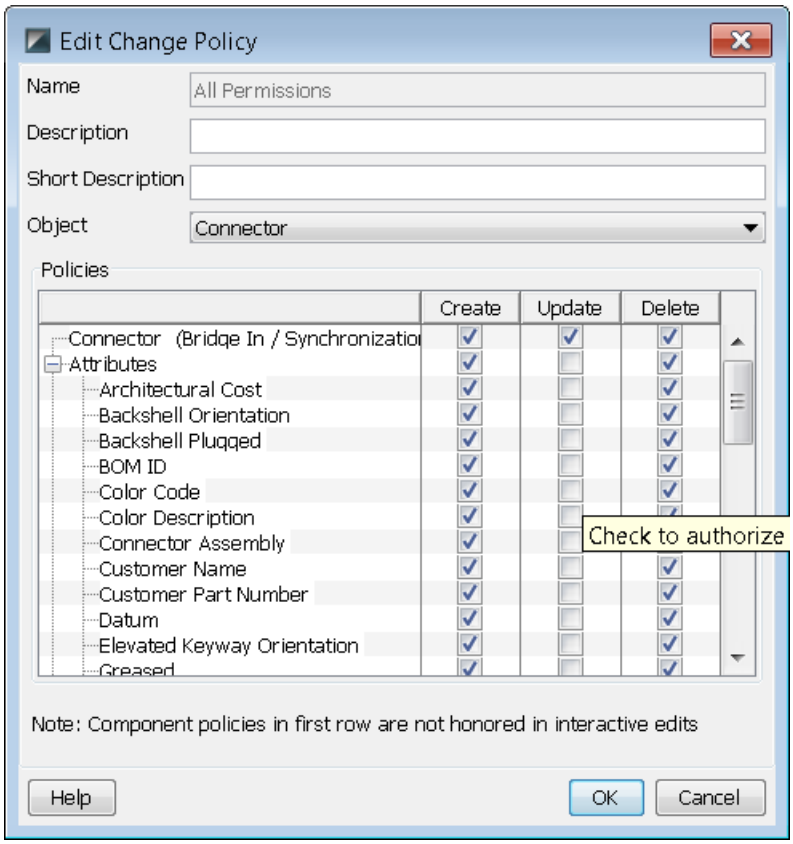
Close the dialog.

k. In the application window on hovering over the user icon at right bottom corner of the window as below, a tool tip pops up and displays the change policy chosen by the plugin

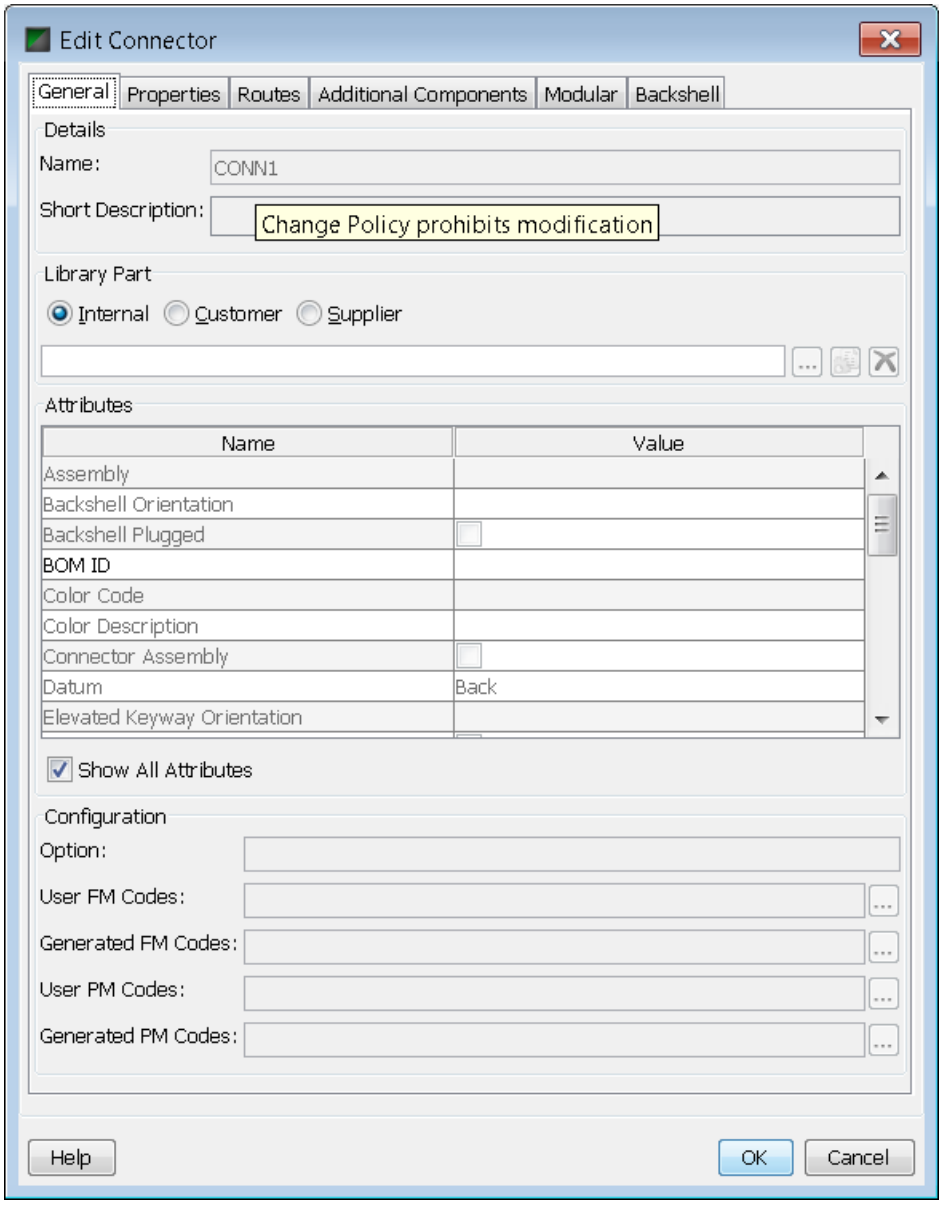



l. In subsequent steps it will be clear on how the chosen change policy is affecting the editing of design objects. For example, assume the connector's change policy has been configured as follows, such that all the attributes are disabled.




When the connector is edited, it will be seen that editing of all the attributes of the connector would become disabled as below:




2) Similarly you can write your own plugin, you have to change the build.xml file accordingly.

# Guidance

Here are some recommendations that will help you …

- The call to *getFilteredChangePolicies*() is issued from the application to the plugin, whenever there is a change in the current context. The IXCurrentContext is defined by the current project, current design and current diagram. This means that, If the window is switched from one diagram to another diagram of same or different designs, a call will be made to getFilteredChangePolicies() from the application.
- The set of change policies supplied to the *getFilteredChangePolicies*() through the parameter *availablePolicies* contains system defined policies as well as the change policies that are defined in the current project.
- If there are any modifications to the change policies they would not be immediately reflected unless they are refreshed by the Refresh policies action.
- If the session's user or his roles have been assigned change policies, the change policy decided by the Plugin takes the priority and is honored for the current session.

# Examples

In this section, an example is presented for the change policy selector plugin. The data that were received in ***getSelectedChangePolicy()***are printed in the console. This sample plugin sorts the given list of policies and chooses the first policy in the sorted list.
This example can be found in the following location within the Capital installation:

**doc/plugin/examples/Java/src/com/example/plugin/changepolicy/**

```
package com.example.plugin.changepolicy;

import com.mentor.chs.plugin.IXCurrentContext;
import com.mentor.chs.plugin.changepolicy.IXChangePolicy;
import com.mentor.chs.plugin.changepolicy.IXChangePolicySelector;
import com.mentor.chs.plugin.user.IXDomain;
import com.mentor.chs.plugin.user.IXRole;
import com.mentor.chs.plugin.user.IXUser;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class ChangePolicySelector implements IXChangePolicySelector
{
        public IXChangePolicy getSelectedChangePolicy(IXCurrentContext context,
                Map<String, ? extends IXChangePolicy> policies)
        {
                displayDetails(context);
                return choosePolicy(policies, context);
        }

        public Set<IXChangePolicy> getFilteredChangePolicies(IXCurrentContext context,
                Map<String, IXChangePolicy> policies)
        {
                displayDetails(context);
                return new HashSet<IXChangePolicy>(policies.values());
        }

        public String getDescription()
        {
                return "Select a Change Policy based on the current context";
        }

        public String getName()
        {
                return "Sample Change Policy Selector";
        }

        public String getVersion()
        {
```

```
        return "0.1";
}

/**
 * Alphabatically sorts the given set of policies from 'policies' and returns the first
        entry in the sorted list.
 *
 * @param policies - The map of changes policies vs their name
 * @param context - The application's current context.
 *
 * @return - Returns a change policy chosen from the set of policies from 'policies'
 *
 */
private IXChangePolicy choosePolicy(Map<String, ? extends IXChangePolicy> policies,
        IXCurrentContext context)
{
        IXChangePolicy retPolicy = null;
        if (policies == null || policies.isEmpty()) {
                System.out.println("No policies to choose from");
                return null;
        }
        List<IXChangePolicy> sorted = new ArrayList<IXChangePolicy>(policies.values());
        Collections.sort(sorted, new Comparator<IXChangePolicy>()
        {
                @Override public int compare(IXChangePolicy o1, IXChangePolicy o2)
                {
                        return o1.getName().compareTo(o2.getName());
                }
        });
        System.out.println(" -- Choices --");

        retPolicy = sorted.get(0);
        for (IXChangePolicy cp : sorted) {
                System.out.println(cp.getName());
        }

        return retPolicy;
}

/**
 * This method displays the context information sent from the application.
 *
 * @param context - the context obtained from the application.
 *
 */
private void displayDetails(IXCurrentContext context)
{
        System.out.println("Current Build List:  " + context.getActiveBuildList());
        System.out.println("Current Project:     " + context.getCurrentProject());
        System.out.println("Current Design:      " + context.getCurrentDesign());
        System.out.println("Current Diagram:     " + context.getCurrentDiagram());
        IXUser user = context.getCurrentUser();
        if (user != null) {
                System.out.println("Current User:        " + user);
                System.out.println("Current User [CP]:   " + user.getChangePolicy());
                for (IXDomain domain : user.getDomains()) {
                        System.out.println("    Domain:   " + domain.getName());
                }
                for (IXRole role : user.getRoles()) {
                        System.out.println("    Role:   " + role.getName());
                }
```

```
                }
        }
}
```